IRYSS Platform Technical Plan

Marketplace · Content Platform with live shoppable streaming · Shopify Connector

Prepared by Samy

Date 20 July 2026

Status Draft for review

Supersedes parts of IRYSS Marketplace Technical Stack and IRYSS Content Platform — see §2 and Appendix A

Contents

1. Scope and how to read this
2. Research findings that change the source documents
3. Locked decisions
4. Repository and workspace structure
5. Service inventory
6. DNS, subdomains and routing
7. Data and storage architecture
8. Identity, authentication and SSO
9. Multi-storefront context model
10. Marketplace core
11. Content Platform, including live streaming
12. Shopify Connector
13. Shared contracts and event taxonomy
14. Infrastructure, environments and CI/CD
15. Observability and security
16. Build sequencing and team
17. Week 1 critical path
18. Risk register
19. Open items
20. Recommended changes to the locked stack

1. Scope and how to read this

This plan covers three applications that together form the IRYSS platform:

Application	Role	Source doc
Marketplace	B2C + B2B commerce, Admin/Brand/Reseller portals, catalogue, orders, search	Marketplace Technical Stack, Build Timeline
Content Platform	YouTube-style content library **plus live shoppable streaming**	IRYSS Content Platform
Shopify Connector	Brand catalogue import, reseller product export	IRYSS Shopify Connector

All three live in **one git repository, one Turborepo workspace**, deploy as **independent Cloud Run services**, and talk to each other only through defined API boundaries.

The plan is written A–Z, but the **build is staged** (§16). Specifying everything up front is what makes the staging safe: the contracts, subdomains, auth model and data boundaries are decided once, so the second and third teams inherit them instead of negotiating them.

Two conventions used throughout:

- **VERIFY** — an assumption that must be confirmed in Week 1 before it becomes load-bearing. Every one of these is listed again in §19.
- **CORRECTION** — this plan deliberately departs from one of the source documents. Rationale is always given.

2. Research findings that change the source documents

Six findings materially affect the architecture. All were verified against primary sources (vendor docs, the actual GitHub repositories, npm registry) in July 2026.

2.1 Bunny Stream cannot do live streaming

Bunny's own FAQ states **"RTMP streams are currently not supported."** There is no RTMP or SRT ingest in Bunny Stream. It is a VOD product: TUS resumable upload → transcode → adaptive-bitrate HLS → token auth → DRM (Enterprise tier).

> **CORRECTION.** Content Platform §10 states "Live streaming — Bunny supports live ingest." This is incorrect. Two third-party review sites claim Bunny supports RTMP; they contradict Bunny's own FAQ and product documentation and should be disregarded.

Because live streaming is a V1 requirement (§3), a second video vendor is required. See §11.3.

2.2 Mercur 2.x is block-based, not fork-and-modify

Mercur **2.0** (released 17 March 2026) replaced the monolithic fork-and-modify model with a **block architecture** modelled on shadcn/ui: the CLI copies source into your project from a registry, rather than you forking the upstream repo. Current version is **2.2.0** (9 July 2026). MIT licensed.

> **CORRECTION.** Marketplace Technical Stack §3 says "IRYSS will fork/clone the relevant Mercur codebase into the IRYSS GitHub environment... The existing Mercur backend, B2C storefront, Admin Panel and Vendor Panel code paths should be preserved as working foundations." That describes **Mercur 1.x**. In 2.x, forking discards the block registry's re-pull path — the framework's principal advantage — and converts every upstream fix into a manual rebase.

The old starter repositories (`mercur-api-starter`, `mercur-storefront-starter`, `mercur-vendor-starter`, `mercur-admin-starter`) are **archived**. The B2C storefront now lives in a separate repository: `mercurjs/b2c-marketplace-storefront`.

2.3 Mercur 2.x already generates a Turborepo

`create-mercur-app@2.2.0` scaffolds:

```
apps/admin/    apps/vendor/    packages/api/
turbo.json     blocks.json     tsconfig.base.json
```

`packages/api` is the Medusa application — `medusa-config.ts` and `instrumentation.ts` live there, not at the repo root.

> This answers the original question directly. The concern was "will Mercur fit inside a Turborepo?" It **is** a Turborepo containing a Medusa app. The real work is reconciling **its** layout with the one proposed in Technical Stack §2, not creating a workspace from scratch. See §4.

2.4 Mercur issue #909 was fixed in 2.x only, with no 1.x backport

Confirmed at `github.com/mercurjs/mercur/issues/909`. Opened 4 May 2026, closed 15 May 2026.

Failure chain as reported: 1. `splitAndCompleteCartWorkflow` does not create Medusa's standard `order_cart` link for the seller orders it creates. 2. Medusa's `processPaymentWorkflow` uses the presence of `order_cart` to decide whether a cart is already complete. With the link absent, the payment webhook **always** attempts to re-complete an already-completed cart. 3. Re-completion fails on inventory conflict → compensation runs → `createOrderSetStep`'s compensation **hard-deletes** `order_set` **rows** without cascading to link tables → orphaned rows, silent data loss. 4. `compensatePaymentIfNeededStep` then **refunds the already-captured payment**.

Closed by a Mercur collaborator with "This issue is addressed inside new Mercur v2." A community member requested a 1.x backport and received none.

Consequence: Mercur 2.x is not optional. On 1.x this bug is live, unpatched, and silently destroys order records while refunding captured money.

2.5 Mercur 2.2.0 reshapes the seller product model

Breaking changes in 2.2.0: - Store API products are now **master products**, with **offers** replacing seller-owned product models. - **Cart line items reference offer IDs, not variant IDs.** - Attributes migrated from Mercur's custom logic to native Medusa product options.

This directly reshapes the seller/brand data model that Build Timeline assigns to Dev 2, and it changes how the Connector writes products (§12).

2.6 Medusa on Cloud Run has three specific hazards

`MEDUSA_WORKER_MODE=server|worker|shared` exists and is the correct scaling primitive. But:

(a) The worker cannot be a Cloud Run Job. Jobs are run-to-completion; the Medusa worker is a long-lived BullMQ consumer with scheduled jobs. It must be a Cloud Run **service** with `--no-cpu-throttling` and `--min-instances=1`. Without CPU-always-allocated, Cloud Run throttles the CPU the moment a response is sent and the queue consumer starves; with scale-to-zero, scheduled jobs simply never fire.

(b) Cold starts. A community measurement put Medusa cold start at 13–14s on Cloud Run. Medusa's maintainers state the framework "is built with the assumption that the server is always on." `--min-instances=1` is required on the API service too. (The measurement is v1-era; no published v2 Cloud Run figure exists — VERIFY in Week 1 with a real deploy.)

(c) `.medusa/server` **runs a nested** `npm install`**.** `medusa build` emits `.medusa/server/` with its **own generated** `package.json`; production runs `cd .medusa/server && npm install && npm run start`. That install resolves against the generated manifest, **not** the workspace root — so `workspace:*` dependencies **resolve in dev and fail in production**.

This is the most dangerous finding for this specific plan, because Month 1's entire strategy is "define typed contracts first, everything builds against them." Mitigation in §4.3.

3. Locked decisions

#	Decision	Choice	Rationale
D1	Live streaming	**V1, shoppable live commerce**	Commercial driver. Supersedes Content Platform §10 (V2).
D2	Storefront addressing	**Subdomains** — `it.iryss.com`, `fr.iryss.com`	One zone, one wildcard cert, cookies shareable on `.iryss.com`, new market = 1 DNS record + 1 DB row.
D3	Live video vendor	**Cloudflare Stream Live** (ingest + live HLS), **Bunny Stream** retained for VOD	Bunny cannot do live (§2.1). Cloudflare is already in the stack for DNS/WAF — no new vendor relationship. Automatic live→VOD recording.
D4	Marketplace foundation	**Mercur 2.x, blocks-native**	#909 fixed (§2.4); blocks stay re-pullable; Turborepo comes for free (§2.3).
D5	Middleware	**Medusa-native, inside `packages/api`**	Ronan's position. No second framework, no hop on hot commerce paths, avoids the `.medusa/server` trap.
D6	Build sequencing	**Relay** — Marketplace → Content Platform → Connector	Matches both source docs' own staffing assumptions. ~7 devs peak.
D7	Root domain	**`iryss.com`**, owned	Single Cloudflare zone.
D8	Package manager	**pnpm**	Only manager with first-party Medusa support and a dedicated troubleshooting doc.
D9	Node	**20 LTS**, pinned	Medusa `engines: >=20`; Mercur generated apps `>=20`.

Note on D5 and the email thread. Ronan was right that the middleware belongs in the Medusa codebase. The original NestJS argument was made to support real-time signaling and WebSockets, and that requirement is real — but it belongs in the **Content Platform** (§11.4), not the middleware. Both positions are satisfied: NestJS is used for the Content Platform and the Connector, exactly as Ronan agreed, and the middleware stays Medusa-native.

4. Repository and workspace structure

4.1 Approach

We do **not** build a Turborepo and drop Mercur into it. We **generate the Mercur workspace with `create-mercur-app` and grow IRYSS inside it.** This preserves `turbo.json`, `blocks.json` and `tsconfig.base.json` as upstream expects, honouring Technical Stack §35: "Do not break Mercur's working build assumptions just to force a clean-looking workspace."

Blocks are pulled via the Mercur CLI into `packages/api/src`, `apps/vendor/src` and `apps/admin/src` per the aliases in `blocks.json`. Block-sourced files are committed but kept in identifiable directories so a future re-pull produces a readable diff.

4.2 Layout

```
iryss-platform/                      ← single git repo, single Turborepo
├── turbo.json
├── blocks.json                      ← Mercur block registry aliases
├── tsconfig.base.json
├── pnpm-workspace.yaml
├── .npmrc                           ← hoist patterns, see 4.4
├── .nvmrc                           ← 20
│
├── packages/
│   ├── api/                         ★ MERCUR / MEDUSA CORE — upstream layout, do not rename
│   │   ├── medusa-config.ts
│   │   ├── instrumentation.ts
│   │   └── src/
│   │       ├── api/
│   │       │   ├── middlewares/     ← THE IRYSS MARKETPLACE MIDDLEWARE (D5)
│   │       │   │     storefront-context.ts
│   │       │   │     rbac.ts
│   │       │   │     cache-scope.ts
│   │       │   │     request-validation.ts
│   │       │   ├── store/  admin/  vendor/
│   │       ├── modules/             ← IRYSS domain modules (reseller, storefront-config)
│   │       ├── workflows/
│   │       ├── subscribers/         ← OpenSearch indexing, enrichment triggers, events
│   │       └── links/
│   │
│   ├── contracts/                   ★ PUBLISHED, not workspace-linked — see 4.3
│   │                                  types · Zod schemas · event taxonomy
│   ├── ui/                          shared design system, portal shells, tables, forms
│   ├── auth/                        session/role/permission helpers
│   ├── api-client/                  typed client for api.iryss.com
│   ├── config/                      env loading + Zod validation
│   └── test-utils/
│
├── apps/
│   ├── admin/                       ← Mercur admin (upstream) + IRYSS design → Admin Portal
│   ├── vendor/                      ← Mercur vendor panel → BRAND PORTAL
│   ├── b2c-storefront/              ← from mercurjs/b2c-marketplace-storefront
│   ├── b2b-storefront/              ← custom; no Mercur foundation exists
│   ├── reseller-portal/             ← custom; reseller data model from day one
│   ├── payload-cms/
│   ├── content-platform/            ← NestJS
│   │     src/api/  src/worker/  src/ws-gateway/
│   ├── shopify-connector/           ← NestJS
│   │     src/api/  src/worker/
│   └── shopify-embedded-app/        ← Shopify React Router template + Polaris Web Components
│
└── infrastructure/                  ← Terraform
      environments/{dev,staging,prod}/
      modules/{cloud-run,cloud-sql,redis,opensearch,dns,secrets,monitoring}/
```

4.3 `packages/contracts` is published, not linked — and this is not optional

Per §2.6(c), `medusa build` produces `.medusa/server/` with its own `package.json` and runs a nested `npm install` at deploy. A `workspace:*` dependency **will resolve in development and fail in production**.

Month 1 of the Build Timeline is built entirely on shared typed contracts. If those contracts are workspace-linked into `packages/api`, the failure surfaces at the first production deploy of the commerce backend — the worst possible place and time.

Resolution:

- `packages/contracts` is versioned and **published to Google Artifact Registry's npm repository** by CI on merge to `main`.
- `packages/api` consumes it as a **real semver dependency** (`"@iryss/contracts": "1.4.2"`).
- All other workspace members may link it normally (`workspace:*`) — they have no nested-install step.
- CI blocks a `packages/api` build if its pinned `@iryss/contracts` version is behind `main`.

This costs one publish step and makes the failure structurally impossible rather than latent.

4.4 Required workspace configuration

Root `.npmrc` — Medusa admin extensions require a single shared instance of these packages; without the hoist patterns the admin panel breaks at runtime:

```
public-hoist-pattern[]=*@medusajs/*
public-hoist-pattern[]=@tanstack/react-query
public-hoist-pattern[]=react-i18next
public-hoist-pattern[]=react-router-dom
```

Additional rules:

- Root `package.json` must declare `packageManager`.
- `@medusajs/framework` **must be a** `peerDependency` (not `devDependency`) in any workspace package whose compiled output imports it. devDependencies are not shipped, so a mistake here fails only in production.
- Pin every `@medusajs/*` package to one exact version via root `pnpm.overrides`. Mercur itself pins all of them to 2.17.2.
- `.medusa/` is gitignored.
- `medusa build` only compiles `.ts`/`.js`. JSON and other assets need an explicit `postbuild` copy step.
- Turbo tasks that drive Medusa must `cd` into `packages/api` — the Medusa CLI resolves relative to CWD, not workspace root. `outputs: [".medusa/**", "dist/**"]`.

4.5 Deleted from the original stack document

Removed	Why
`apps/workers`	Medusa needs no separate worker app — it is the same image deployed twice with `MEDUSA_WORKER_MODE`. Content Platform and Connector keep their own NestJS workers (separate services, separate databases).
`apps/mercur-middleware`	Per D5, moved into `packages/api/src/api/middlewares/`.
`apps/mercur-core`	Renamed to `packages/api` to match Mercur's generated layout (D4).
`packages/types`, `packages/validation`, `packages/events`	Merged into `packages/contracts` — they are one versioning unit and must be published together (§4.3).

5. Service inventory

Fifteen Cloud Run services. Each has its own Dockerfile, its own deploy, its own scaling profile.

#	Service	Source	Notes
1	`api-server`	`packages/api`	`MEDUSA_WORKER_MODE=server`, `DISABLE_MEDUSA_ADMIN=false`, `min-instances=1`
2	`api-worker`	`packages/api` (**same image**)	`MEDUSA_WORKER_MODE=worker`, `DISABLE_MEDUSA_ADMIN=true`, **`--no-cpu-throttling`**, `min-instances=1`
3	`b2c-storefront`	`apps/b2c-storefront`	Serves **all** country storefronts; market resolved from `Host`
4	`b2b-storefront`	`apps/b2b-storefront`	Gated
5	`admin-portal`	`apps/admin`	
6	`brand-portal`	`apps/vendor`	
7	`reseller-portal`	`apps/reseller-portal`	
8	`payload-cms`	`apps/payload-cms`	
9	`content-api`	`apps/content-platform`	NestJS REST
10	`content-worker`	`apps/content-platform` (same image)	`--no-cpu-throttling`, `min-instances=1`
11	`content-ws`	`apps/content-platform` (same image)	**WebSocket gateway** — `--session-affinity`, `min-instances=1`, long request timeout
12	`connector-api`	`apps/shopify-connector`	NestJS REST + Shopify webhook receiver
13	`connector-worker`	`apps/shopify-connector` (same image)	`--no-cpu-throttling`, `min-instances=1`
14	`shopify-app`	`apps/shopify-embedded-app`	Shopify App URL + OAuth callback
15	`gtm-server`	Google-provided image	Server-side GTM container

Pattern: services 1–2, 9–11 and 12–13 are each one image deployed multiple times with different entrypoints/env. This keeps build time and registry cost down and guarantees the worker runs identical code to the API.

Every worker service needs `--no-cpu-throttling` and `--min-instances=1`. This is a cost decision as much as a technical one: those containers are always on. Cloud Run's scale-to-zero economics do not apply to queue consumers. Budget accordingly.

6. DNS, subdomains and routing

6.1 Production

Single Cloudflare zone: **`iryss.com`**. Cookies scoped to `.iryss.com`.

Subdomain	Service	Public	Notes
`iryss.com`	`b2c-storefront`	✔	Geo-select / redirect to market
`www.iryss.com`	→ `iryss.com`	✔	301
`it.iryss.com`	`b2c-storefront`	✔	`storefront=IT` resolved from Host
`fr.iryss.com`	`b2c-storefront`	✔	`storefront=FR`
`<mkt>.iryss.com`	`b2c-storefront`	✔	Future market = 1 DNS record + 1 DB row
`trade.iryss.com`	`b2b-storefront`	✔	Account-gated
`admin.iryss.com`	`admin-portal`	—	Cloudflare Access / IP allowlist
`brands.iryss.com`	`brand-portal`	—	Authenticated
`resellers.iryss.com`	`reseller-portal`	—	Authenticated
`api.iryss.com`	`api-server`	✔	Medusa store/admin/vendor API + IRYSS middleware
`cms.iryss.com`	`payload-cms`	—	Admin UI authenticated; delivery API read-only
`content.iryss.com`	`content-api`	✔	Content Platform REST
`ws.iryss.com`	`content-ws`	✔	**WSS** — live chat, presence, product pinning
`live.iryss.com`	Cloudflare Stream	✔	Live HLS playback — **VERIFY** custom domain support
`vod.iryss.com`	Bunny Stream	✔	VOD playback, token auth — **VERIFY** custom hostname
`cdn.iryss.com`	Bunny CDN pull zone	✔	Images, optimiser, resizing
`connect.iryss.com`	`connector-api`	✔	Connector API + **Shopify webhook endpoint**
`shopify.iryss.com`	`shopify-app`	✔	**Shopify App URL + OAuth redirect URI**
`t.iryss.com`	`gtm-server`	✔	Server-side GTM, first-party context

6.2 The two Shopify subdomains — why both are needed

This was asked for specifically, and it is genuinely two hostnames doing two different jobs:

- **`shopify.iryss.com`** is what Shopify itself knows about. It is registered in the Shopify Partner dashboard as the **App URL** and **OAuth redirect URI**. It serves the Polaris UI that renders

inside the merchant's Shopify admin. Its job is narrow, per Connector §2: install, OAuth, access-code validation, connected status.

- `connect.iryss.com` is the Connector backend. It receives **Shopify webhooks** (product create/update/delete, inventory, orders, app-uninstall, and the three mandatory compliance webhooks), and it serves the Connector screens embedded in the Brand, Reseller and Admin portals.

Keeping them apart matters because the App URL is registered with Shopify and is painful to change, whereas the backend will evolve. It also means webhook traffic and merchant UI traffic scale independently.

6.3 Live ingest — correction

RTMPS ingest does not get an IRYSS subdomain. Cloudflare Stream Live ingest is a Cloudflare-owned endpoint (`rtmps://live.cloudflare.com:443/live/`), and RTMPS ingest cannot be fronted by a CNAME. Brands receive a **Cloudflare-provided ingest URL plus a per-stream key issued by the Content Platform**. Only playback is on an IRYSS hostname.

Earlier sketches in this discussion showed a `stream-in.iryss.com`. That was wrong; it is corrected here.

6.4 Non-production

```
<service>.stg.iryss.com     staging
<service>.dev.iryss.com     development
```

Certificate note: a `*.iryss.com` wildcard does **not** cover `*.stg.iryss.com` — wildcards match one label only. Three certificates are required, one per level.

6.5 SSL termination

Technical Stack §27 flags this as a decision the engineering lead must make deliberately. Making it here:

Cloudflare proxied (orange cloud) → Full (strict) → Google Cloud Load Balancer → Cloud Run.

Why GCLB rather than Cloud Run domain mappings, since "Cloudflare is already in front, do we need it?" is a fair challenge and the answer is yes:

- Google's own documentation states Cloud Run domain mapping is **"in the preview launch stage"** and **"not production-ready,"** and is not recommended for production due to latency.
- Domain mappings work in **only 10 regions**, support **no wildcard certificates**, take up to **24 hours** to provision a certificate, and can only map to `/`.
- **Cloud Armor and IAP attach to the load balancer, not the service** — without a GCLB neither is available.
- Critically: with a GCLB you set Cloud Run ingress to **"Internal and Cloud Load Balancing."** Without that, clients reach services directly on their default `*.run.app` URLs, **bypassing Cloudflare, the WAF, and every access control**. There is no other way to close that door.

Cost is roughly **$18–20/month** for the first five forwarding rules plus ~$8/TB processed. That is cheap insurance against an open `run.app` back door into admin and API services.

- Cloudflare holds the edge certificate; GCLB holds a Google-managed certificate; the hop between them is TLS-verified.
- WAF, bot protection, rate limiting and DDoS controls live at Cloudflare.
- **Cloudflare must not cache** dynamic commerce, account, cart, checkout, admin or API responses (Technical Stack §27). Cache rules explicitly bypass `api.`, `admin.`, `brands.`, `resellers.`, `connect.`, `ws.`, and any `/cart`, `/checkout`, `/account` path.
- `ws.iryss.com` needs Cloudflare **WebSocket support enabled**, and must not sit behind rules that buffer or terminate long-lived connections.
- `t.iryss.com` is proxied so server-side GTM is genuinely first-party.

7. Data and storage architecture

7.1 Relational — Google Cloud SQL for PostgreSQL

Three logical databases. Both new-application docs specify a "dedicated PostgreSQL database" each, and that boundary is right: it prevents the Content Platform and Connector from ever becoming a second source of commerce truth.

Database	Owner	Contents
`iryss_marketplace`	`packages/api`	Medusa/Mercur core
`iryss_payload`	`payload-cms`	CMS content — **separate database, see below**
`iryss_content`	`content-api`	Channels, videos, playlists, live sessions, Bunny/CF asset refs, analytics rollups
`iryss_connector`	`connector-api`	Store connections, product mappings, sync history, retry/webhook state, errors

> **CORRECTION to Technical Stack §13**, which says Postgres stores "the main operational data for Mercur and, where configured, Payload." **Payload must not share a database with Medusa.** Medusa's own official Payload integration guide provisions a separate `PAYLOAD_DATABASE_URL`. Three concrete hazards: (a) two independent migration systems — Medusa uses MikroORM, Payload uses Drizzle, and Payload's docs warn its `push` mode must not be mixed with manual migrations; (b) table-name collisions — both want `users`, `products`, `orders`, and Payload's docs explicitly warn against overlap; (c) pool contention on a single `max_connections` ceiling. Payload's `schemaName` option exists but is **marked experimental** with a history of bugs. Same instance is fine; same database is where it breaks.

Instance topology: - **dev / staging** — one Cloud SQL instance, four databases. Cost efficiency. - **production** — `iryss_marketplace` + `iryss_payload` on one instance; `iryss_content` + `iryss_connector` on a second. Rationale: the marketplace instance carries checkout and must not contend with video-processing or bulk-sync write bursts. - **Production must be regional (HA) with point-in-time recovery enabled.** Neither source document mentions HA, backups or DR. For a system holding orders and payments this is not optional.

No cross-database joins, ever. The Content Platform and Connector reach commerce data only through `api.iryss.com`.

7.2 Connection pooling

Technical Stack §13 correctly notes that Cloud SQL Auth Proxy provides secure connectivity, **not** pooling. Concretely:

- Each Cloud Run instance holds its own pool. Cloud Run fans out. `max_connections` is a hard ceiling that is easy to hit.
- Per-instance pool size: **single digits** (start at 5).

- `--max-instances` **must be set on every service** and the arithmetic checked: `Σ(max_instances × pool_size) < max_connections × 0.8`.
- Use the Cloud SQL connector with a **lazy** refresh strategy — background refresh is throttled by Cloud Run CPU throttling.
- Automatic reconnection is mandatory; the platform drops idle connections.
- If the arithmetic stops working, add **PgBouncer** or Cloud SQL Managed Connection Pooling. Do not raise `max_connections` as the fix.

7.3 Cache and queues — Google Memorystore for Valkey

> **CORRECTION to Technical Stack §15**, which specifies Aiven for Valkey. **Google Memorystore for Valkey is now GA with a 99.99% SLA**, 1–250 node clusters, zero-downtime scaling, managed backups, cross-region replication and **native Private Service Connect**. Aiven's PSC support on GCP is still labelled **"early availability"**, is per-service, and is unsupported for BYOC. Since every consumer of this cache runs on Cloud Run inside GCP, first-party PSC and a 99.99% SLA outweigh vendor consistency. **Aiven is retained for OpenSearch** (§7.4), where Google has no managed first-party equivalent — so this is not a vendor removal, only a reallocation.

Medusa v2 needs Redis/Valkey for five distinct purposes, each configured separately. This is easy to under-provision:

Purpose	Configuration
Session store	`projectConfig.redisUrl` — **do not wire** `connect-redis` **by hand**; Medusa does it internally. Without `redisUrl`, sessions are in-memory and are lost on every Cloud Run scale event.
Cache	`@medusajs/cache-redis`
Event bus	`@medusajs/event-bus-redis`
Workflow engine	`@medusajs/workflow-engine-redis`
Distributed locking	`@medusajs/locking-redis` — **required for multi-instance production.** Neither source document mentions it. Without it, concurrent Cloud Run instances can race on the same workflow.

> **CORRECTION to Technical Stack §16**, which says "the implementation can use `connect-redis`." `connect-redis` is still a dependency of `@medusajs/framework`, but it is wired internally. You set `redisUrl`; you do not install it yourself. Note also that the session connection is **not shared** with the four module connections above — each is configured independently.

Single Valkey cluster, **namespaced by application**, per both new docs' explicit requirement that video and connector jobs not collide with marketplace jobs:

```
mkt:*        marketplace cache, sessions, BullMQ (Medusa)
content:*    content platform cache, BullMQ
connect:*    connector cache, BullMQ
live:*       live session state, presence, viewer counts
```

Cache keys must include full storefront context — storefront, region, locale, currency, sales channel, permission scope (Technical Stack §6). A cache key that omits any of these can serve French pricing into the Italian storefront. This is enforced by a helper in `packages/contracts`, not by convention.

Sessions are Redis-backed so Cloud Run scaling doesn't lose them.

7.4 Search — Aiven OpenSearch

One cluster, multiple indices:

Index	Documents
`products`	Marketplace catalogue, with `storefront[]`, `region`, `locale`, `sales_channel`, `visibility` fields on every doc
`content`	Videos as their own content type (title, description, channel, brand, category, collection, playlist, product links, visibility, processing state, viewer access)
`brands`	Brand/seller discovery

Per Technical Stack §18: **one shared search architecture with storefront fields indexed on documents, filtered at query time** — not isolated per-country indices. This lets Admin run consolidated search while each storefront still sees only its own catalogue.

OpenSearch is never authoritative. On conflict, Postgres wins. Indexing runs through Medusa subscribers → BullMQ → indexing workers, with retry, failed-index logging and status tracking.

7.5 Media

Asset	Vendor	Path
Product images, brand visuals	**Bunny** Storage + CDN + Optimizer	`cdn.iryss.com`
VOD video	**Bunny Stream**	TUS upload → transcode → ABR HLS → `vod.iryss.com`, token auth
Live video	**Cloudflare Stream Live**	RTMPS ingest → live HLS → `live.iryss.com`, signed URLs
Live recording → VOD	Cloudflare → worker → Bunny	See §11.5

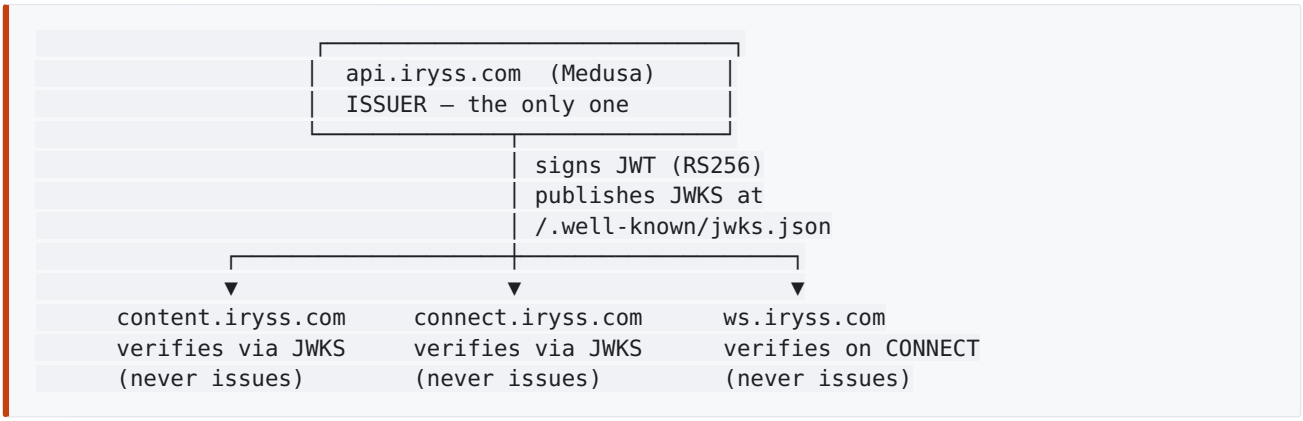
Per Technical Stack §20: Bunny is the delivery and optimisation layer, **not** the image-management workflow. Upload rules, validation, crop standards, approval, ordering, alt-text and brand asset rules are IRYSS application logic.

8. Identity, authentication and SSO

Both new documents require it — Connector §10: "A signed-in marketplace user is recognised consistently across the Connector, marketplace, and Content Platform" — but neither specifies the mechanism. Specifying it here.

8.1 Model

`api.iryss.com` **(Medusa) is the sole identity provider.** The Content Platform and Connector are **resource servers**. They never authenticate a user themselves and never hold a password.



- **RS256 asymmetric signing.** Resource servers verify with the public key; only Medusa holds the private key. A compromised Content Platform cannot mint marketplace tokens.
- JWKS cached with a short TTL so key rotation propagates without redeploys.
- Session cookie scoped to `.iryss.com` (viable because of D2 — this is a concrete dividend of choosing subdomains over ccTLDs).

8.2 Token claims

```
{
  "sub": "usr_...",
  "actor_type": "customer|b2b_user|brand_user|reseller_user|admin|storefront_manager",
  "storefronts": ["IT", "FR"],
  "brand_id": "sel_...",
  "reseller_id": "res_...",
  "permissions": ["catalogue:read", "orders:write", "content:upload"],
  "consent": { "analytics": true, "marketing": false }
}
```

`storefronts` and `permissions` are what make server-side RBAC enforceable in the resource servers without a callback to Medusa on every request.

8.3 Authorisation rules

- **Coarse checks are local.** Content Platform and Connector enforce `actor_type`, `storefronts` and `permissions` straight from verified claims.

- **Fine-grained commerce authority is delegated.** "May this reseller download this product's video?" requires live product-allocation state and is asked of `api.iryss.com`. Both source docs already specify this path.
- **Service-to-service calls** (Connector → Content Platform for video retrieval) use a separate service credential, never a user token.
- Answers to delegated checks are cached in Valkey with short TTL, keyed including permission scope.

8.4 Enforcement discipline

Technical Stack §29: **frontend restrictions are never enough.** The Build Timeline is explicit that RBAC is a server-side verification task, not a UI task. Concretely, every role × region × storefront × section × function combination gets an automated permission test in `packages/test-utils`, run in CI. Hidden navigation is not evidence of anything.

9. Multi-storefront context model

9.1 Generic by construction

Italy and France are **configured rows, not code**. A `storefront` record carries:

```
code              IT
domain            it.iryss.com
region_id         reg_...           (Medusa region)
sales_channel     sc_...            (Medusa sales channel)
locale            it-IT
currency          EUR
default_lang      it
catalogue_visibility_rules
content_context
operational_owner
```

Adding Germany is a DNS record plus a row. No code change. This is Build Timeline's explicit requirement and it is worth defending against pressure to hardcode.

9.2 Resolution and fail-closed behaviour

`storefront-context.ts` resolves context on every relevant request from: request `Host` → route → locale → storefront record → region → sales channel → authenticated permissions.

It must fail closed. If storefront, domain, locale, region, sales channel or permission context cannot be resolved safely, the request is **rejected or routed to a controlled error state**. It must never silently default to another storefront — that leaks the wrong products, prices, content, analytics or operational data (Technical Stack §6).

Resolved context is then applied consistently to Medusa queries, OpenSearch queries, Payload content queries, PostHog events, admin views and workflow actions.

9.3 Central catalogue, never duplicated

One product = one core commerce record. Storefront-specific availability, visibility, pricing, content, localisation and brand eligibility are applied through Medusa configuration, custom fields, sales-channel/region rules and middleware-enforced context. **No per-country product tables.**

9.4 Leakage testing

A named regression suite that asserts, for every pair of storefronts, that no product, price, content block, search result, analytics record or admin view crosses between them. This runs in CI, not as a manual Month-4 exercise.

10. Marketplace core

10.1 Foundation

Mercur 2.2.0 on Medusa 2.17.2, blocks-native (D4). Mercur core supplies sellers, commissions, payouts, split carts, order groups, native Stripe Connect marketplace payouts, 35+ admin pages, 19+ vendor pages, plus optional blocks (reviews, product import/export, team management, wishlist, notifications, requests, vendor chat).

Blocks IRYSS is likely to adopt: **requests** (reused for the Connector's request-access workflow, Connector §9.21), **product import/export**, **notifications**, **team management**.

10.2 The offers model — a real data-model change

Per §2.5, Mercur 2.2.0 makes Store API products **master products** with **offers** replacing seller-owned product models, and **cart line items reference offer IDs, not variant IDs**.

Consequences to design around from day one:

- Brand product management (Brand Portal) operates on **offers against master products**, not on independently owned products.
- The Connector's SKU-matching logic (Connector §9.3) matches to **master products** and creates/updates a **brand's offer** — not a new product per brand.
- Reseller product allocation attaches to an offer.
- OpenSearch product documents must model the master-product/offer relationship, or search will show duplicates or miss sellers.

This is a Week-1 spike item, not a Month-3 discovery.

10.3 Split checkout — issue #909

Fixed upstream in 2.x, but **verify against the exact pinned version in Week 1** (this was already Ronan's instruction and it stands). Test, with evidence:

1. `order_cart` link **is** created for every seller order in `splitAndCompleteCartWorkflow`.
2. Payment webhook correlation does **not** attempt to re-complete a completed cart.
3. Compensation uses **soft delete** (`deleted_at`), not hard delete, on `order_set`.
4. No auto-refund of captured payment on inventory conflict.

Deliberately force an inventory conflict during a split checkout and confirm no money moves and no rows vanish. If any of the four fails, that is a launch blocker and the plan needs re-scoping in Week 1 rather than Month 4.

10.4 Portals

Portal	Foundation	Notes
Admin	Mercur admin	**One application.** Global admin sees consolidated data and filters into a storefront; storefront managers see only permitted storefronts. Enforced server-side.
Brand	Mercur vendor panel	Extends the seller model where it aligns.
Reseller	**None — custom**	Reuses the shared dashboard shell, UI components, auth pattern, tables, forms, filters. **Wired to a reseller-specific data model and workflows from day one** — it is not a re-skinned Brand Portal.

10.5 B2C / B2B storefronts

- **B2C** — from `mercurjs/b2c-marketplace-storefront`, redesigned to the IRYSS Figma, preserving working product/session/cart/checkout/order logic. **One deployment serves every market**; the market comes from `Host`.
- **B2B** — no Mercur foundation exists. Custom Next.js on the same backend, contracts, deployment model and design system. Its own trade navigation, account-gated flows, pricing behaviour and product visibility rules.

10.6 AI Product Enrichment

A controlled background workflow, **not** a separate AI platform. Structured output (Zod-validated) → server-side business-rule validation → write back to Medusa product custom fields. BullMQ queue with retry. Status field: `pending | complete | failed | approved | edited | rejected`. Downstream systems (search, SEO, Connector) read **from Medusa**, never from a separate AI store.

Image alt-text and classification are **steps inside this workflow** via a vision-model call — not a separate system, and not something Bunny does.

11. Content Platform

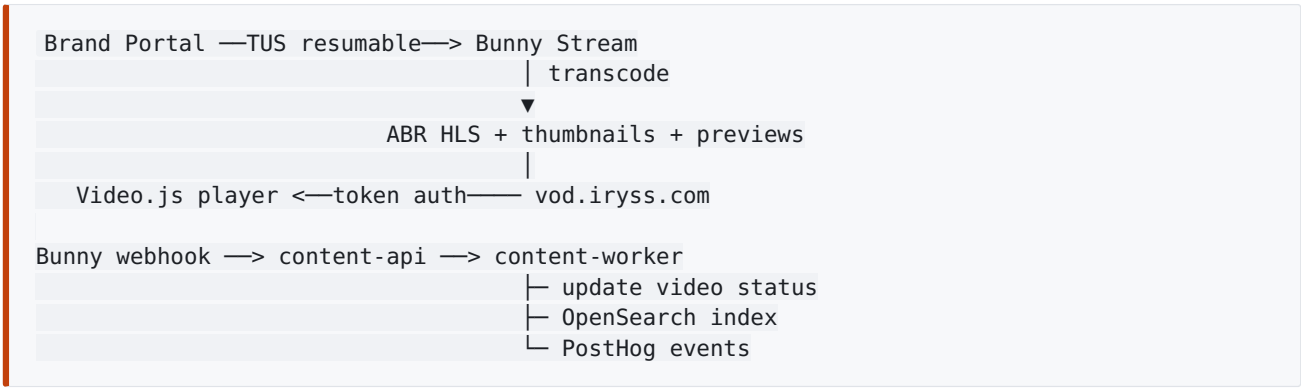
NestJS on Node 20, own Postgres, own BullMQ namespace, own Cloud Run services, screens embedded in the Brand, Reseller and Admin portals.

11.1 Responsibilities

Channels (brand + IRYSS), videos, playlists, collections, publishing state, visibility and download rules, product-linked media, playback data, analytics, Bunny/Cloudflare asset references — **and live sessions**.

It stores the **product reference**; the product itself stays in the marketplace. All live commerce data (price, stock, availability, reseller access) is fetched through `api.iryss.com`.

11.2 VOD pipeline (per source doc, unchanged)



Lifecycle: `Draft → Uploading → Processing → Ready → Published`, plus `Failed` and `Unpublished`. Once a channel is published, individual videos auto-publish when processing completes **and** required metadata is present. No per-video manual approval step.

Player: **Video.js** — adaptive playback, thumbnails, chapters, playback speed, hover-scrub previews (Bunny-generated preview assets), product-link interface, up-next.

11.3 Live pipeline (new — D1, D3)

```
brand (OBS / mobile)
   │  RTMPS + stream key issued by content-api
   ▼
Cloudflare Stream Live          ← ingest endpoint is Cloudflare-owned (§6.3)
   │  live HLS, signed URLs
   ▼
live.iryss.com ──> Video.js player
                        ▲
                        │  real-time overlay
                  ws.iryss.com  (NestJS WebSocket gateway)
                        ├─ pinned product card push
                        ├─ live chat + moderation
                        ├─ presence / viewer count
                        └─ "buy now" → marketplace cart

stream ends ──> Cloudflare recording ──> content-worker ──> Bunny Stream
                                                            (shoppable replay)
```

11.4 The WebSocket gateway

This is the service the original NestJS argument was actually about, now correctly located.

- Separate Cloud Run service (`content-ws`) from the same image as `content-api`.
- **Cloud Run session affinity enabled**; long request timeout; `min-instances=1`.
- **Valkey pub/sub for fan-out.** Cloud Run scales horizontally — a chat message arriving at instance 3 must reach viewers on instances 1, 2 and 4. Socket state must never be instance-local. This is the single most common way live chat breaks in production.
- Auth on `CONNECT` via the marketplace JWT (§8), then per-message authorisation from claims.
- Live session state, presence and viewer counts in Valkey under `live:*`.
- Rate limiting per connection; moderation actions (mute, ban, delete message) available to brand and admin roles.

11.4.1 Cloud Run's 60-minute hard cap — a real constraint on live commerce

Verified against Google's documentation, and it is the most serious infrastructure finding after the `.medusa/server` trap:

Constraint	Reality
Max request/ connection duration	**60 minutes (3600s), hard ceiling.** Default is 5 minutes. WebSockets are treated as ordinary HTTP requests and the connection is **killed at the ceiling regardless of activity.**
Session affinity	**"Best effort" only.** Google states plainly that WebSocket requests "could still potentially end up at different instances."
Billing	Instances with open connections bill **CPU-allocated continuously**. An idle socket still costs money.
HTTP/2	Must **not** enable end-to-end HTTP/2 on a WebSocket service.

What this means concretely: a 90-minute live shopping event will forcibly disconnect every viewer at the 60-minute mark. Not degrade — disconnect. Any design that assumes a stable socket for the duration of a stream is wrong on Cloud Run.

This is survivable, but only if it is designed for from the first line of code:

1. **Mandatory client auto-reconnect** with backoff and state resync (`reconnecting-websocket` or Socket.IO's built-in handling). Google prescribes this explicitly.
2. **Zero instance-local state.** Every socket must be able to land on any instance at any moment. Pinned products, chat backlog, presence and viewer counts all live in Valkey; the gateway is stateless. This was already the plan for scale-out — the 60-minute cap makes it non-negotiable rather than merely correct.
3. **Reconnection must be invisible to the viewer.** The video stream is HLS over plain HTTP and is unaffected; only the overlay drops. Done properly the user sees nothing.
4. **Test explicitly for it** — a >60-minute live session with product pinning and chat is a required test case, not an edge case.

VERIFY in Week 1 with a real load test: reconnection behaviour at the cap, Valkey pub/sub fan-out across instances, and cost at expected concurrency. **If reconnect-at-60-minutes proves unacceptable to the business** (long-form live events are core to the format), the fallback is **GKE Autopilot for `content-ws` only** — everything else stays on Cloud Run. Decide in Week 1; it is cheap now and expensive in Month 6.

11.5 Live → VOD

When a stream ends, Cloudflare produces a recording. A worker transfers it into Bunny Stream, attaches it to the channel, carries over the pinned-product timeline so the replay stays shoppable, and indexes it in OpenSearch. Long-tail delivery then runs on Bunny's cheaper egress — the reason for keeping two vendors rather than moving VOD to Cloudflare.

11.6 Security

Playback happens through IRYSS-controlled pages so viewer access is checked before the player renders. Protected VOD uses **Bunny tokenized playback + domain restrictions**; live uses **Cloudflare signed URLs**. DRM stays V2.

11.7 Analytics

PostHog events: `video_viewed`, `video_watch_progress`, `video_completed`, `video_product_clicked`, `video_saved`, `video_downloaded`, `video_shared`, plus new live events: `live_stream_started`, `live_viewer_joined`, `live_product_pinned`, `live_product_clicked`, `live_chat_message`, `live_stream_ended`.

Bunny supplies VOD delivery performance; Cloudflare supplies live delivery performance; PostHog supplies business context. Brand and Admin reporting blends all three.

11.8 Scope note

Content Platform §11 quotes **2.5–3 weeks for V1 with 3 developers** — for a V1 that explicitly **excluded live streaming**. Shoppable live adds: a second video vendor integration, stream-key management, the WebSocket gateway with pub/sub fan-out, live chat, moderation tooling, live product pinning, presence, and live→VOD recording. Realistic revised estimate: **6–7 weeks with 3–4 developers.** This should be stated plainly to Ronan rather than absorbed silently.

12. Shopify Connector

NestJS, own Postgres, own BullMQ namespace, plus the embedded Shopify app. Two directions, one application.

12.0 The embedded app stack is out of date in the source document

> **CORRECTION to Connector §7 and §8**, which specify "Shopify's official app template, built around Express, Vite, and React" and **Polaris**. That describes `shopify-app-template-node`, which is legacy — its last twelve months of commits are CODEOWNERS churn and Dependabot merges.

Current state, verified against the Shopify repos:

Template	Status
`shopify-app-template-react-router`	**Current.** What `shopify app init` scaffolds today.
`shopify-app-template-remix`	Maintained, but its README explicitly redirects to React Router — "As of React Router v7, Remix and React Router have merged."
`shopify-app-template-node` (Express/Vite/React)	Effectively abandoned

Use `shopify-app-template-react-router`. Stack: React Router 7+, TypeScript, Vite, Prisma session storage (point it at `iryss_connector`), App Bridge, and **Polaris Web Components**. Server package is `@shopify/shopify-app-react-router`. It ships a Cloud Run deployment guide, which suits us directly.

Polaris React is deprecated — `Shopify/polaris-react` was archived read-only, superseded by Polaris Web Components. Existing apps keep working, but new work should not start on the React implementation.

Next.js was considered and rejected. There is no official Shopify Next.js template and no first-party adapter; every starter is a community project. Building it in Next.js to match the rest of the estate would mean owning the OAuth, session, webhook and token-exchange plumbing that the React Router template provides free, with no first-party support path. The consistency gain is not worth it for a single embedded app whose scope is deliberately narrow.

This does introduce React Router as a fourth frontend framework alongside Next.js (storefronts), Vite/React (Mercur panels) and the NestJS backends. That is an acceptable, contained cost: the embedded app is small, it lives inside Shopify's admin rather than IRYSS's, and it must follow Shopify's conventions to feel native and to pass app review.

12.1 Boundaries

- Marketplace reads/writes go **only** through `api.iryss.com`. The Connector never touches the marketplace database.
- Product video is **retrieved** from the Content Platform; the Connector stores no video.

- Shopify-specific code stays inside the Shopify integration layer (OAuth, API clients, webhooks, bulk operations, rate-limit handling, Shopify errors). This preserves the seam for a future non-Shopify connector.
- The Connector stores only: connections, mappings, sync history, retry state, webhook state, errors.

12.2 Brand import

Access code (issued after IRYSS approval) → Shopify OAuth → **GraphQL Bulk Operations** catalogue pull → SKU match against marketplace → create/link → category mapping (confident auto, uncertain via AI proposal + brand confirm, confirmed mappings saved and reused) → **trigger marketplace AI enrichment** → brand correction → **existing Admin Product Approvals screen** → live.

Ongoing changes via webhooks. Bulk queries structured as several purpose-built queries (max 5 connections, nesting depth 2 per query); API versions 2026-01+ allow 5 concurrent bulk queries and 5 concurrent bulk mutations per shop — **VERIFY** limits against the active API version at implementation.

12.3 Reseller export

Browse approved catalogue → select (creates/confirms allocation) → set retail markup over marketplace wholesale price → push as **native Shopify products** with approved media, Content Platform video, AI-generated SEO fields, collection placement → bulk mutations where supported.

Hard rules: - **No IRYSS branding, watermark or marketplace reference** on any pushed product. - **Never touch the reseller's theme, homepage, layout or navigation.** - A push either lands fully formed or fails cleanly and retries. No half-created products. - Disconnect **unpublishes**, never deletes.

Reseller Shopify sales forward inventory updates and sales-event records into the marketplace. **The Connector does not implement stock concurrency logic** — it forwards the signal; the marketplace inventory and reservation system resolves conflicts.

12.4 Reliability

- **Idempotency everywhere**, keyed on Shopify's webhook ID for webhooks and on explicit idempotency references for jobs. Applies to webhooks, scheduled jobs, manual retries, imports, pushes, inventory updates, sales events, deletion and disconnect.
- Webhook endpoint does the minimum: verify signature → queue raw payload → return 200. All processing in workers.
- Per-store queue scheduling so one rate-limited store cannot starve worker capacity for others.
- Shopify's three **compliance webhooks** (customer data request, customer data erasure, shop data erasure) implemented as light handlers — the Connector holds no customer checkout data by design.

12.5 Launch dependency resolved

Technical Stack §3 leaves open whether the Reseller Portal launches with Shopify push disabled/stubbed or with a limited slice pulled forward. Under D6 (relay), the Connector lands **after** marketplace launch, so:

> **The Reseller Portal ships at marketplace launch with Shopify connection UI present but disabled**, showing a clear "coming soon" state. The reseller-specific data model, allocation workflow and product-selection flows are built in full — only the Shopify push is inert. When the Connector ships, it activates without portal rework.

13. Shared contracts and event taxonomy

13.1 `packages/contracts`

Single published package (§4.3) holding:

- TypeScript types for every cross-service payload
- **Zod schemas** for API requests/responses, form inputs, worker job payloads, webhook payloads, tracking events, enrichment outputs, configuration objects, **and environment variables**
- The event taxonomy (§13.3)
- Cache-key builders that enforce storefront-context inclusion (§7.3)

Environment variables are validated at application startup in every service. A missing or malformed variable fails the container immediately rather than at first use.

13.2 Contract-first working

Weeks 1–2 lock data models, API contracts and routing. Storefronts, portals, search, tracking and workers then build against **typed contracts and mocked responses** rather than waiting for real endpoints. This is what makes seven developers parallel in Month 1, and it is the single control most worth protecting.

Breaking changes to `packages/contracts` require a major version bump and an explicit consumer-update PR. CI blocks `packages/api` builds that pin a stale version.

13.3 Event taxonomy

Controlled names, consistent properties, validated payloads. Frontends may trigger events but **must not** scatter independent tracking calls; everything routes through the middleware where identity attachment, consent checks, enrichment and forwarding rules are applied server-side.

Marketplace: `product_viewed`, `product_saved`, `search_performed`, `filter_applied`, `brand_viewed`, `add_to_cart`, `checkout_started`, `order_completed`, `recommendation_clicked`, `brand_portal_product_uploaded`, `reseller_product_selected`.

Content: `video_viewed`, `video_watch_progress`, `video_completed`, `video_product_clicked`, `video_saved`, `video_downloaded`, `video_shared`.

Live: `live_stream_started`, `live_viewer_joined`, `live_product_pinned`, `live_product_clicked`, `live_chat_message`, `live_stream_ended`.

Connector: `connector_store_connected`, `connector_import_started`, `connector_import_completed`, `connector_product_pushed`, `connector_sync_failed`.

13.4 Identity convention

Anonymous visitor ID, session ID, logged-in customer ID, B2B account/user ID, brand user ID, reseller user ID, portal user ID — with consent-aware state attached at event level. Defined once in `packages/contracts` and used by every application, so behaviour stitches cleanly across anonymous browsing, logged-in activity, portal actions, video watching and live participation.

13.5 Consent and privacy

Built **alongside** tracking in Month 1–2, never retrofitted. Controls analytics consent, marketing consent, cookie categories, PostHog capture, GTM firing permissions, consent auditability and regional compliance. Consent state travels in the JWT (§8.2) so server-side event routing can honour it without a lookup.

14. Infrastructure, environments and CI/CD

14.1 Environments

Env	GCP project	Domain	Notes
dev	`iryss-dev`	`*.dev.iryss.com`	Shared Cloud SQL instance, small OpenSearch
staging	`iryss-staging`	`*.stg.iryss.com`	Production-shaped, reduced capacity
production	`iryss-prod`	`*.iryss.com`	Full topology

Separate GCP projects, not separate namespaces — it makes IAM boundaries and cost attribution real.

14.2 Terraform

Manages Cloud Run services, Cloud SQL, load balancers, IAM roles, service accounts, Secret Manager entries, Artifact Registry, DNS, networking and monitoring. No manual console changes; drift is treated as an incident.

14.3 CI/CD

```
GitHub Actions
  ├─ install (pnpm, cached)
  ├─ lint · typecheck · unit tests (turbo, affected-only)
  ├─ publish @iryss/contracts on version change   ← gate for packages/api
  ├─ integration tests
  ├─ Playwright E2E on critical paths
  ├─ dependency + security scan
  ├─ docker build per affected app
  ├─ push → Google Artifact Registry
  └─ deploy → Cloud Run  (prod requires manual approval)
```

Medusa Dockerfile specifics: - Run `medusa build` **and the nested** `.medusa/server` `npm install` **in the image build stage**, never at container start — doing it at start compounds an already-slow cold start. - `predeploy` runs `medusa db:migrate`. - Build `--admin-only` separately if the admin is ever hosted apart from the API.

14.4 Secrets

Google Secret Manager for: database credentials, Medusa/Mercur secrets, Payload secrets, OpenSearch credentials, Bunny keys, **Cloudflare Stream credentials**, PostHog keys, email provider keys, server-side GTM secrets, Shopify app credentials, finance/shipping/tax integration credentials, AI provider keys, webhook signing secrets, JWT signing keys.

Least-privilege service accounts per Cloud Run service. No service account may read another application's secrets.

15. Observability and security

15.1 Observability

Google Cloud Logging, Cloud Monitoring, Error Reporting, plus **Sentry** for frontend and backend error tracking.

Alerting from day one on: worker queue depth and failed jobs, OpenSearch indexing failures, payment webhook failures, **live stream health and WebSocket connection counts**, Connector sync failures, API latency, Cloud Run instance health, database connection saturation, failed deployments.

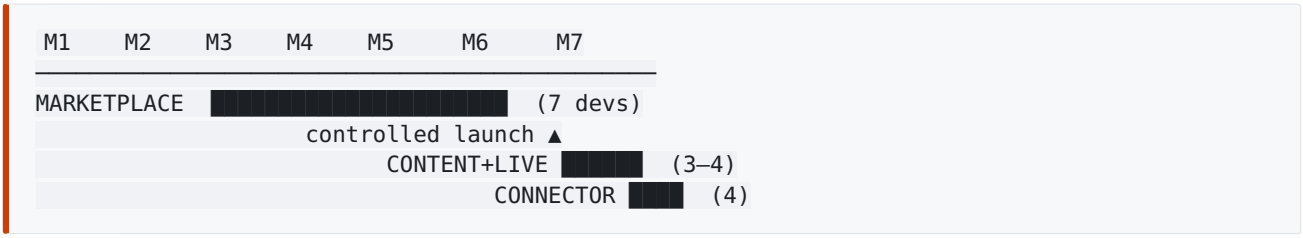
Structured logs carry the storefront context and request ID so a single request can be traced across storefront → API → worker → search.

15.2 Security

- **Permissions enforced at the backend/API layer.** Frontend restrictions are never sufficient.
- Controlled CORS origins per subdomain — not a wildcard.
- Rate limiting at Cloudflare and in-application.
- Webhook signature validation on every inbound webhook (Shopify, Bunny, Cloudflare, payment provider).
- Admin action audit logging.
- Cloudflare Access or IP allowlist on `admin.iryss.com`.
- Least-privilege service accounts; no shared credentials across applications.
- Secrets never in environment files in the repo; all via Secret Manager.

16. Build sequencing and team

16.1 Shape (D6 — relay)



Each team arrives already knowing the middleware contracts, portal shells, shared UI, deployment pipeline and testing patterns — which is precisely why the source docs quote 3 weeks and 4.5 weeks for the later applications. Those figures are only credible under a relay.

16.2 Marketplace — 7 developers

Allocation follows Build Timeline with two amendments:

Dev	Ownership	Amendment
1	Technical lead / architect — Mercur decisions, Turborepo boundaries, standards, Week-1 spike, RBAC review, checkout risk, release control. **Hands-on and coding.**	Adds: contracts publishing pipeline, `.medusa/server` build strategy
2	Mercur/Medusa commerce core — regions, sales channels, seller/brand model, catalogue, pricing, inventory, cart/ checkout/orders, returns, split-order verification	**Adds the 2.2.0 offers/order-groups model (§10.2) — materially larger than the doc assumes**
3	Middleware — storefront context, RBAC enforcement, typed contracts, validation, response shaping, event routing, webhooks, retry/idempotency, integration boundaries, cache scoping	**Now inside `packages/api` (D5), not a separate app**
4	B2C + B2B storefronts	
5	Admin Portal	
6	Brand Portal + Reseller Portal	
7	Search, enrichment, media, workers, test automation	

Devs 3 and 7 carry the heaviest integration load — the concern raised in the original email correspondence. Dev 1 must actively protect them from becoming the critical path.

16.3 Later phases

- **Content Platform + live** — 3–4 devs, **6–7 weeks** (revised from 2.5–3, per §11.8).
- **Connector** — 4 devs as two pairs (brand import / reseller export) working in parallel, 4–4.5 weeks. Excludes Shopify app-review time, which must be started early because it is externally gated.

16.4 Honest total

Phase	Duration
Marketplace to controlled launch	~4 months
Marketplace stabilised commercial release	~5 months
Content Platform + live streaming	+6–7 weeks
Shopify Connector	+4–4.5 weeks
Full platform	**~7–7.5 months**

The marketplace figures track the original estimate. The growth over the ~6.5-month naive sum is live streaming, which was priced as V2 in the source document and is now V1.

17. Week 1 critical path

Week 1 exists to prove assumptions before the build accelerates. Everything below either de-risks a decision in this plan or unblocks an externally-gated dependency.

17.1 Mercur / Medusa spike

- [ ] `create-mercur-app@2.2.0` scaffold; confirm generated Turborepo layout
- [ ] Add one IRYSS app and one shared package; confirm `turbo` tasks work end to end
- [ ] **Prove the `.medusa/server` contracts strategy** — publish `@iryss/contracts` to Artifact Registry, consume from `packages/api`, deploy to Cloud Run, confirm it resolves in production (§4.3)
- [ ] Pull two blocks; confirm the re-pull diff is readable
- [ ] **Model the 2.2.0 offers / master-product relationship** and confirm it supports the IRYSS brand and reseller workflows (§10.2)
- [ ] **Re-verify issue #909** against the exact pinned version — all four assertions in §10.3, with a deliberately forced inventory conflict
- [ ] Regions, sales channels, storefront metadata, catalogue visibility per storefront
- [ ] Admin panel, vendor panel and storefront extension methods
- [ ] Workflow/subscriber extension pattern, custom fields, RBAC extension approach
- [ ] OpenSearch indexing hook points

17.2 Infrastructure

- [ ] Three GCP projects, Terraform skeleton
- [ ] Cloud SQL, Valkey, OpenSearch provisioned
- [ ] Artifact Registry (docker **and npm** repositories)
- [ ] Cloudflare zone, DNS records per §6, **SSL termination configured per §6.5**
- [ ] **Set Cloud Run ingress to "Internal and Cloud Load Balancing" on every service** — verify `*.run.app` URLs are unreachable directly (R14)
- [ ] First Cloud Run deploy of `api-server` + `api-worker`; **measure real cold start** (§2.6b)
- [ ] Verify worker CPU-always-allocated actually processes scheduled jobs
- [ ] **Configure all five Medusa Redis purposes** — `redisUrl` plus cache, event bus, workflow engine and **locking** modules (§7.3)
- [ ] Cloud SQL regional HA + PITR enabled on the production instance; separate `iryss_payload` database created

17.3 Live streaming spike — the highest-unknown item

- [ ] Cloudflare Stream Live account; RTMPS ingest from OBS → HLS playback working
- [ ] **VERIFY** custom domain support for `live.iryss.com`
- [ ] **VERIFY** Cloudflare live-to-VOD recording export path into Bunny

- [ ] **Load-test WebSockets on Cloud Run** — scale-out, best-effort affinity, Valkey pub/sub fan-out across instances (§11.4)
- [ ] **Test a >60-minute session explicitly** — confirm reconnection is invisible to the viewer and no state is lost. Decide Cloud Run vs GKE Autopilot for `content-ws` **this week** (§11.4.1)
- [ ] Confirm Cloudflare Stream Live pricing at expected concurrency, and WebSocket instance cost at expected viewer counts

17.4 External dependencies — start now, they gate everything

- [ ] Shopify Partner account + app registration (**app review is externally gated — start early**)
- [ ] Payment provider / Stripe Connect
- [ ] Tax provider / Avalara
- [ ] Odoo / accounting
- [ ] Shipping provider / Sendcloud
- [ ] Email provider
- [ ] PostHog, Bunny (storage + CDN + Stream), consent provider selection

Technical Stack §1 warns these "can become the real critical path if delayed." That warning is correct and should be treated as a schedule risk, not a checklist item.

17.5 Contracts

- [ ] `packages/contracts` v0.1.0 published
- [ ] Storefront context, RBAC claims, product, order, video, live-session and connector-sync contracts drafted
- [ ] Event taxonomy agreed
- [ ] Mock middleware responses available so Devs 4, 5, 6 can start

18. Risk register

#	Risk	Impact	Mitigation
R1	`.medusa/server` nested install breaks shared contracts	**Critical** — surfaces at first production deploy of the commerce backend	Publish `@iryss/contracts` (§4.3). **Prove in Week 1.**
R2	Mercur #909 not actually fixed in pinned version	**Critical** — silent order data loss + refunded captured payments	Week-1 verification with forced inventory conflict (§10.3). Launch blocker if it fails.
R3	**Cloud Run's 60-minute hard cap disconnects every live viewer mid-event**	**High — confirmed, not hypothetical.** A 90-minute live shopping event drops all sockets at 60 min	Mandatory client auto-reconnect, zero instance-local state, >60-min test case. GKE Autopilot fallback for `content-ws` only if the business rejects reconnection (§11.4.1)
R4	2.2.0 offers model doesn't fit IRYSS brand/ reseller workflows	High — reshapes catalogue, Connector and search	Week-1 modelling spike (§10.2)
R5	Live streaming scope creep past 6–7 weeks	High	Live product pinning, chat and moderation are the V1 line. Vertical reframing, social simulcast, DRM and Whisper stay V2.
R6	Cloudflare Stream Live pricing at scale	Medium	Model cost at expected concurrency in Week 1; Bunny retains VOD long tail specifically to contain this
R7	Storefront leakage between markets	High — wrong prices/products/ data exposed	Fail-closed context resolution (§9.2), context in every cache key (§7.3), automated pairwise leakage suite (§9.4)
R8	DB connection exhaustion from Cloud Run fan-out	High	Small pools, `max-instances` caps, documented arithmetic (§7.2), PgBouncer as the remedy
R9	Shopify app review delays Connector launch	Medium	Register the app in Week 1, months before it is needed
R10	External provider setup becomes critical path	High	All initiated Week 1 (§17.4)
R11	Contract drift between the three applications	Medium	Single published package, semver, CI gate on stale pins (§13.2)
R12	Blocks diverge from upstream, losing re-pull benefit	Medium	Keep block-sourced files in identifiable directories; review diffs on re-pull; do not refactor block internals casually
R13	Always-on worker services cost more than serverless budget assumed	Medium	Five services require `min-instances=1` + no-CPU-throttling (§5). WebSocket instances bill CPU continuously even when idle. Budget explicitly.
R14	Direct `*.run.app` access bypasses Cloudflare, WAF and all access controls	**High**	Cloud Run ingress set to "Internal and Cloud Load Balancing" on every service; GCLB mandatory (§6.5)

#	Risk	Impact	Mitigation
R15	Payload and Medusa migration systems collide	High if shared DB is adopted	Separate databases from day one (§7.1)
R16	`@medusajs/locking-redis` omitted → workflow races across Cloud Run instances	High	Configure all five Redis purposes explicitly in Week 1 (§7.3)

19. Open items

Every **VERIFY** in this document, collected:

1. Medusa v2 cold-start time on Cloud Run — the 13–14s figure is v1-era and unmeasured for v2 (§2.6b)
2. Native-dependency rebuild behaviour inside `.medusa/server` — no authoritative documentation found (§2.6c)
3. Cloudflare Stream custom domain support for `live.iryss.com` (§6.1)
4. Bunny Stream custom hostname support for `vod.iryss.com` (§6.1)
5. Cloudflare Stream Live → Bunny recording export path (§11.5)
6. Cloud Run WebSocket max connection duration and scale-out behaviour (§11.4)
7. Shopify bulk operation concurrency limits on the active API version (§12.2)
8. Cloud Run worker pools as an alternative to always-on services — no evidence anyone has run a Medusa worker on it (§5)
9. Aiven for Valkey GCP region coverage — no published per-service region list found (moot if C5 is accepted)
10. Whether Cloudflare's orange cloud interferes with Google-managed certificate validation for IAP — community sources only, not Google documentation (§6.5)

Decisions still owed by IRYSS, not by engineering:

1. Consent-management provider selection
2. Transactional email provider selection
3. Whether Mercur's Enterprise tier is needed, or MIT core suffices
4. Live streaming moderation policy — who moderates, what the escalation path is, what the legal position is on user-generated live chat in IT/FR
5. **Is IRYSS spending on paid acquisition at launch?** Determines J1 (server-side GTM) — a ~$200/month decision
6. **Will live events routinely exceed 60 minutes?** Determines whether reconnection-at-the-cap is acceptable or `content-ws` moves to GKE (§11.4.1)

20. Recommended changes to the locked stack

Everything here is a proposed change to Technical Stack, which describes itself as "locked." Each is either forced by a verified fact or is a judgement call flagged as such. Ronan asked for exactly this — "we are open to change if there's a better approach" — so these are put plainly rather than softened.

20.1 Changes forced by verified facts

#	Change	From → To	Why
C1	Live video vendor	Bunny Stream → **Cloudflare Stream Live** (live) + Bunny (VOD)	Bunny's own FAQ: "RTMP streams are currently not supported." Live is impossible on Bunny. (§2.1)
C2	Mercur adoption	Fork-and-modify → **blocks-native, 2.x**	Fork-and-modify describes 1.x. #909 is unpatched in 1.x with no backport. (§2.2, §2.4)
C3	Shared contracts	`workspace:*` → **published to Artifact Registry**	`.medusa/server` nested install cannot resolve workspace protocol. Fails in production only. (§4.3)
C4	Payload database	Shared with Medusa → **own database**	Two migration systems, guaranteed table-name collisions, pool contention. Medusa's own integration guide uses separate databases. (§7.1)
C5	Valkey provider	Aiven → **Google Memorystore for Valkey**	Memorystore is GA with a 99.99% SLA and native PSC; Aiven's GCP PSC is "early availability." All consumers are inside GCP. Aiven retained for OpenSearch. (§7.3)
C6	Medusa Redis config	"use `connect-redis`" → **`redisUrl` + four separate modules**	`connect-redis` is wired internally. **`@medusajs/locking-redis` is required for multi-instance** and appears in neither source document. (§7.3)
C7	Shopify embedded app	Express/Vite/React + Polaris React → **React Router template + Polaris Web Components**	The Express template is effectively abandoned; Polaris React is archived read-only. (§12.0)
C8	`apps/workers`	Separate application → **`MEDUSA_WORKER_MODE` on the same image**	Medusa's own scaling primitive. One fewer app and Dockerfile. (§4.5)
C9	Worker deployment	"Cloud Run worker services **or jobs**" → **services only**, `--no-cpu-throttling`, `min-instances=1`	Jobs are run-to-completion; a BullMQ consumer is long-lived. As a Job, scheduled jobs never fire. (§2.6a)
C10	Cloud SQL	Unspecified → **regional HA + point-in-time recovery**	Neither document mentions HA, backup or DR for a database holding orders and payments. (§7.1)

20.2 Judgement calls — worth deciding, not forced

J1 — Defer the server-side GTM container until paid acquisition starts. Technical Stack §31 includes it from launch. Google's guidance is a minimum of 2 always-on instances at ~$45–50 each, so **$90–150/month, plus log storage that can add $100+/month at scale** — before a single ad is bought.

It is genuinely **not** redundant with PostHog, and the document is right to say so: PostHog is product analytics ingestion, sGTM is a first-party proxy that fans events out to **ad platforms** (Meta CAPI, Google Ads enhanced conversions) for attribution and bid optimisation. PostHog does not do that at all.

But the decision axis is "are we spending on paid acquisition at launch?" If not, this is ~$200/month of overhead for a capability with nothing consuming it. The subdomain (`t.iryss.com`), consent plumbing and event taxonomy should all be built in Month 1–2 as planned — only the container is deferred. Standing it up later is an afternoon.

J2 — Evaluate merging the B2C and B2B storefronts into one application. Currently two Next.js apps on the same backend, same contracts and same design system. PLP, PDP, cart and checkout are largely duplicated; only navigation, gating, pricing behaviour and visibility rules genuinely differ — and those are exactly the things the storefront-context resolver already handles per-`Host` (§9.2). `trade.iryss.com` could resolve to a B2B context in the same way `it.iryss.com` resolves to Italy.

Potential saving: several weeks of Dev 4's time and one deployment target. Risk: if the B2B journeys diverge more than expected, unpicking them later is worse than having built them apart. **Decide in Week 1** once the B2B journey designs are actually in front of us — not by assumption now. Same question applies, more weakly, to Brand and Reseller portals: their data models must be separate (both source docs are emphatic and correct), but that does not by itself require two deployments.

J3 — Drop Zustand unless a concrete need appears. Technical Stack §3 lists TanStack Query and Zustand. TanStack Query owns server state; React's own state handles the rest. Adding a global store by default invites server state to leak into it, which is the failure mode TanStack Query exists to prevent. Add it when something actually requires it.

J4 — Consolidate error tracking on Sentry. Technical Stack §30 lists Cloud Logging, Cloud Monitoring, Error Reporting **and** Sentry. Error Reporting and Sentry overlap almost entirely for application errors. Suggest: **Sentry for application errors** across all frontends and backends (better grouping, releases, source maps, and it spans the whole estate uniformly); **Cloud Logging/Monitoring for infrastructure** — Cloud Run health, database pressure, queue depth. Skip Error Reporting rather than maintaining two error inboxes nobody reads.

J5 — Confirm OpenSearch is worth its operational cost at launch. Keeping it, but naming the trade-off. OpenSearch is the right long-term choice — Technical Stack §18's "vector-ready semantic discovery foundation" is real, and Meilisearch/Typesense do not offer it. It is also the most operationally demanding component in the stack, and Mercur ships an **Algolia block** out of the box that would work on day one. If search relevance work slips or the cluster becomes a time sink in Month 2, the Algolia block is a credible interim fallback that does not waste the indexing pipeline — the subscribers, jobs and workers are the same shape either way.

20.3 Explicitly not changed

Worth recording, so it is clear these were considered rather than overlooked:

- **Cloud Run** as the runtime — correct, with the documented exceptions for workers and the open question on `content-ws` (§11.4.1).
- **Turborepo, TypeScript, Zod, TanStack Query, React Hook Form** — all sound.
- **PostHog** from launch — right call; behavioural history cannot be backfilled.
- **Bunny for images and VOD** — correct and cost-effective; only live was impossible.
- **Terraform, Artifact Registry, GitHub Actions, Secret Manager** — standard and correct.
- **Cloudflare for DNS/WAF with no caching of dynamic commerce** — correct, and §27's warning is well-judged.
- **Fail-closed storefront context** — the single best decision in the source document. Preserved verbatim.
- **Vitest/Jest split by package convention** — correct; do not force one runner into `packages/api`.

Appendix A — What changed from the source documents

Source	Original	This plan	Why
Stack §2	`apps/mercur-core`	`packages/api`	Mercur 2.x generated layout (§2.3)
Stack §2	`apps/mercur-middleware`	`packages/api/src/api/middlewares/`	D5
Stack §2	`apps/workers`	Deleted — `MEDUSA_WORKER_MODE`	§4.5
Stack §2	`packages/types`, `validation`, `events`	Merged → `packages/contracts`, published	§4.3
Stack §3	Fork Mercur, preserve code paths	Blocks-native, `create-mercur-app` root	§2.2
Stack §3	Shopify push "disabled or stubbed?" open	Resolved: disabled at launch, portal built in full	§12.5
Stack §13	Postgres stores Mercur "and, where configured, Payload"	Payload gets its **own database**	§7.1
Stack §15	Aiven for Valkey	**Google Memorystore for Valkey**; Aiven kept for OpenSearch	§7.3
Stack §16	"can use `connect-redis`"	`projectConfig.redisUrl` + four separate Redis modules incl. **locking**	§7.3
Stack §17	Workers as "Cloud Run worker services **or jobs**"	**Services only**, `--no-cpu-throttling`, `min-instances=1`	§2.6a
Stack §20	Bunny for media	Bunny VOD + **Cloudflare Stream Live**	§2.1
Stack §27	SSL termination "must be decided"	Decided: CF proxied → Full (strict) → GCLB, ingress locked to LB	§6.5
Stack §30	Cloud Error Reporting **and** Sentry	Sentry for app errors; Cloud for infra (J4)	§20.2
Stack §31	Server-side GTM from launch	**Deferred** until paid acquisition; plumbing built (J1)	§20.2
Connector §8	Shopify template "Express, Vite, React" + Polaris	**React Router template** + Polaris Web Components	§12.0
—	No HA/backup/DR specified	Regional HA + PITR on production Cloud SQL	§7.1
Content §6	Bunny Stream for video	Bunny VOD only; Cloudflare for live	§2.1
Content §10	Live streaming = V2	Live streaming = **V1, shoppable**	D1
Content §11	V1 = 2.5–3 weeks	V1 + live = 6–7 weeks	§11.8
Timeline	7 devs, 4–5 months (marketplace)	Unchanged for marketplace; ~7–7.5 months full platform	§16.4

Source	Original	This plan	Why
—	No auth mechanism specified	RS256 JWT, Medusa as sole issuer, JWKS	§8
—	No live ingest subdomain question	Ingest is Cloudflare-owned; only playback is IRYSS	§6.3